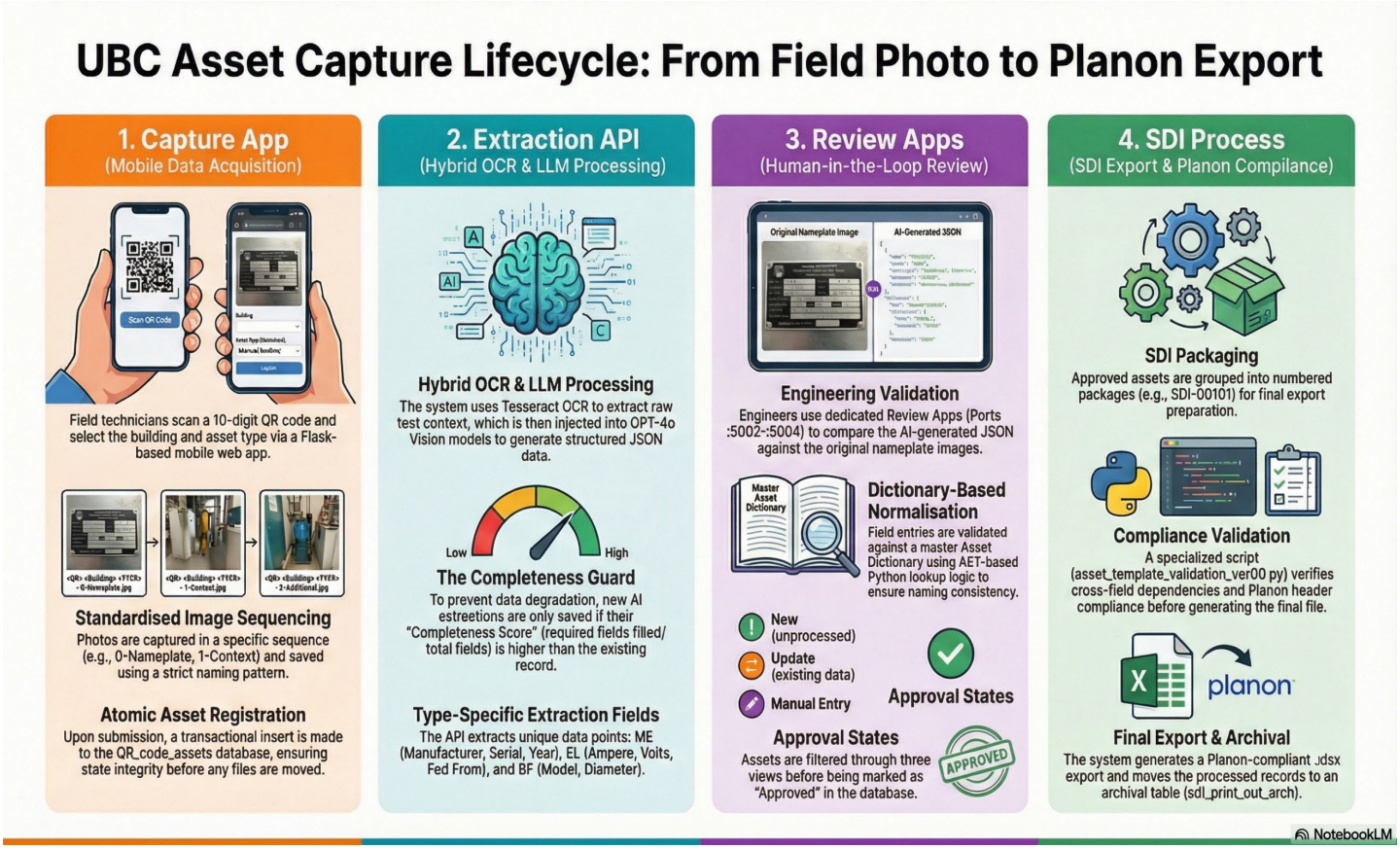
UBC Asset Capture Application — Table of Contents

Current Schema Notes

json_files, sdi_dataset, and sdi_dataset_EL now carry Avg_ai_conf where available. Review and dashboard consumers should read those curated values rather than infer old defaults.
Valid QR IDs should remain unique after normalization. Placeholder IDs should be blocked, and SDI joins should never use numeric coercion on mixed QR formats.
Manual-entry exclusion depends on aligned state across QR_code_assets.Col_process, JSON ExcludeSDI, and QR_codes.sdi.

7. Output Schemas



UBC Asset Capture Application — Technical Documentation

This document provides a complete developer reference for the UBC Facilities Asset Management platform by covering architecture, data flow, coding standards, workflows, special processes, and database schema.

---

1. Platform Overview

1.1. Module Map

Module	Responsibility	Port
Capture App	Mobile field capture, atomic rename service	`:5001`
Extraction API	Hybrid OCR/LLM scripts (ME/EL/BF), JSON output, DB sync	—
Auth Service	Global authentication, shared user DB	—
Review Apps	Human review loop for ME, BF, EL	`:5002` `:5003` `:5004`
Dashboard	Analytics, task launching, FLS CRUD	`:8002`
Dictionary	AST-based Python lookup logic	—
SDI Process	Export packaging & Planon compliance validation	—

1.2. Platform Handoff Flow

Capture App (Photos/SQLite) → Extraction API (JSON) → Review Apps (Overrides/Approvals) → SDI Process (Planon Export)

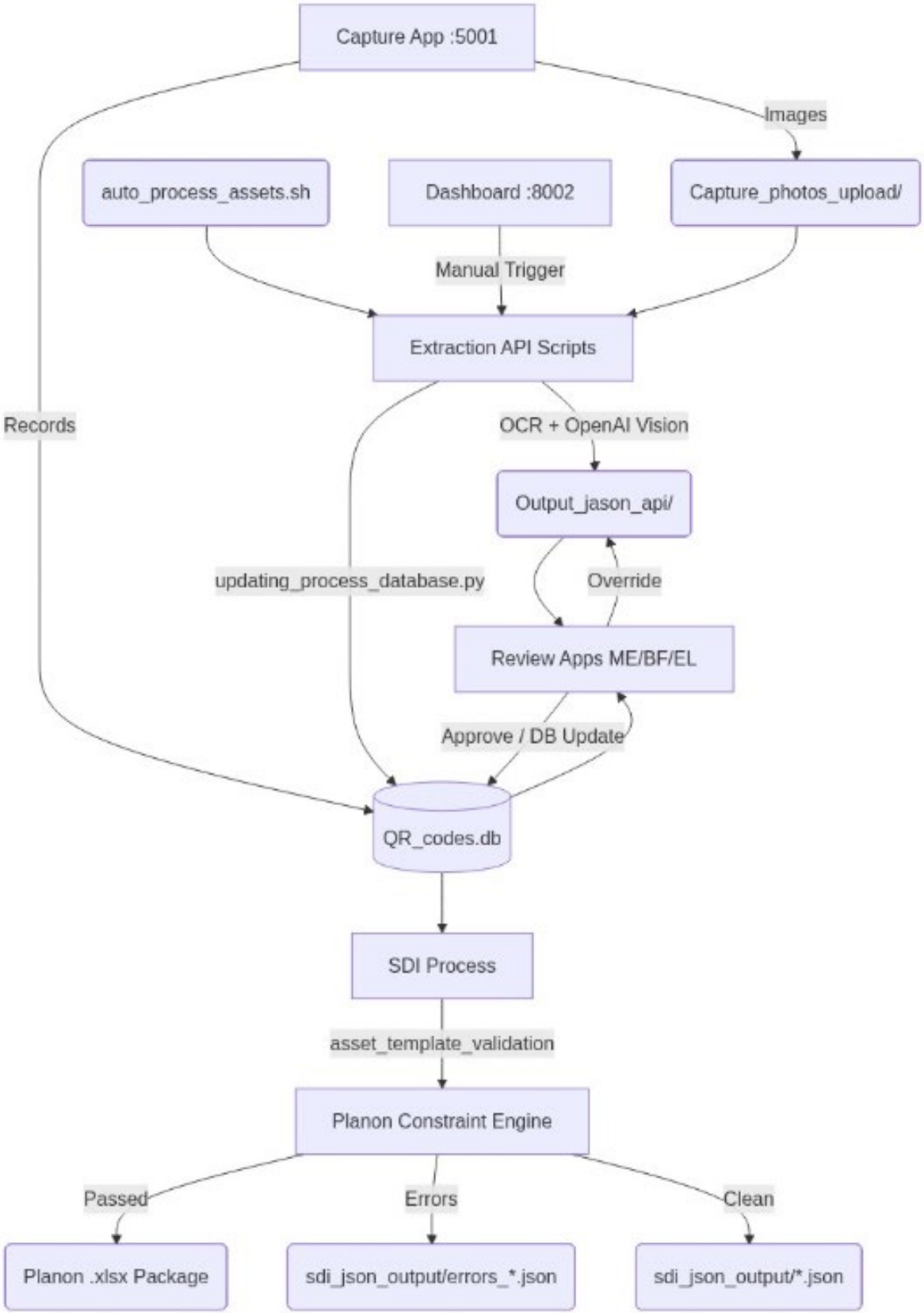
1.3. Environment & Network Topology

- **Host**: Ubuntu Server (`/home/developer/`)
- **Capture App**: `https: /capture.assetcap.facilities.ubc.ca` (`:5001`)
- **ME Review App**: `https: /me-review.assetcap.facilities.ubc.ca` (`:5002`)
- **BF Review App**: `https: /bf-review.assetcap.facilities.ubc.ca` (`:5003`)
- **EL Review App**: `https: /el-review.assetcap.facilities.ubc.ca` (`:5004`)
- **Dashboard**: `https: /dashboard.assetcap.facilities.ubc.ca` (`:8002`)

1.4. File Tracking & Datastores

1. `QR_codes.db` : Primary SQLite DB – tracking data, SDI packaging states, FLS device records, image locations.
2. `User_control.db` : Shared auth persistence (isolated from operational data).
3. `Capture_photos_upload/` : Image storage – pattern: `<QR> <Building> <TYPE> - <SEQ>.jpg` .
4. `Output_jason_api/` : API JSON output – pattern: `<QR>_<TYPE>_<Building>.json` .

1.5. Data-Flow Diagram



2. Global Rules & Conventions

Current Rules (March 2026)

Completeness and AI confidence are discipline-specific. Mechanical counts Technical Safety BC only when sequence -3 exists. Backflow counts Manufacturer, Model, Serial Number, and Diameter. Electrical counts UBC Asset Tag, Ampere, and Supply From only.
EL Avg_ai_conf excludes Volts, Location, and Branch Panel. Blank final fields must not retain stale non-zero confidence values after merge or validation.
The extraction completeness guard applies to extraction rewrites. Human review saves remain allowed to preserve intentional corrections even when they are less complete than an earlier AI pass.

---

1. Do Not Duplicate Logic (DRY)

- **Shared Validators:** All field normalizers live in `validators_shared.py` . Never duplicate.
- **Shared Auth:** Always use the `auth_service` module and `User_control.db` .

2. Completeness Guard

- AI extraction results should only be overwritten if the new completeness score is **≥** the existing JSON. Never damage existing data.
- Formula: `(non-empty required fields / total required fields) × 100`

3. Atomic Updates & State Integrity

- Use `parameter_update_service.py` for all rename/parameter change operations.
- Roll back all operations upon any file system or DB exception.

4. Cross-Platform Paths

- Always resolve paths dynamically: `os.getenv("DEV_PATH", "/home/developer/") + os.path.join()` .

5. Security Guardrails

- Commit NO SECRETS, always use `.env` files.
- Never execute arbitrary strings, use `ast.literal_eval()` for dictionary parsing.
- All SQL queries MUST be parameterized (`?`).

6. Directory Conventions

- **Asset Images:** `Capture_photos_upload/`
- **Output JSONs:** `Output_jason_api/`
- **Shared Database:** `asset_capture_app_dev/data/QR_codes.db`

3. Module Architecture

3.1. Capture App (:5001)

Attribute	Value
Main Script	`app.py` (865 lines)
Framework	Flask + HTML5 MediaDevices API
Key Service	`utils/parameter_update_service.py` (773 lines) – atomic rename engine
Database	`data/QR_codes.db` (19 tables)
Templates	`base.html` , `start.html` , `capture.html`

Core Flow: QR Scan → Select Building/Location/Type → Capture Photos → Submit → `Capture_photos_upload/` + DB records

3.2. Extraction API

Script	Asset Type	Lines	AI Model	OCR Strategy
`API_interface_ME_ver00.py`	Mechanical	3,896	Multi-tier (gpt-5.2 → 5.1 → 4o)	Advanced (region crops + rotation)
`API_interface_EL_ver00.py`	Electrical	688	Single (gpt-4o)	Standard (context injection)
`API_interface_BF_ver00.py`	Backflow	719	Single (gpt-4o)	Extreme Optimized (4 calls/image)
`validators_shared.py`	Shared	152	—	—

Core Flow: `Capture_photos_upload/` → Tesseract OCR + GPT-4o Vision → `Output_jason_api *.json` → `QR_codes.db`

3.2.1. Extracted Fields by Type

Current Scoring Notes

The field tables below describe extracted data, but completeness and confidence now use different discipline-aware subsets. ME uses the core plate fields plus Technical Safety BC only when sequence -3 exists. BF uses Manufacturer, Model, Serial Number, and Diameter. EL completeness uses UBC Asset Tag, Ampere, and Supply From only.
EL confidence excludes Volts, Location, and Branch Panel. Avg_ai_conf should be derived from the included fields only.

ME	EL	BF
Manufacturer	UBC Asset Tag	Manufacturer
Model	Branch Panel	Model
Serial Number	Ampere	Serial Number
Year	Volts	Diameter
UBC Tag	Fed / Fed From	
Technical Safety BC	Location	

3.3. Review Apps

Current Review Behavior

ME, BF, and EL dashboards now expose Avg AI Conf using discipline-aware values, color bands, and a confidence slicer.
Save and approval must preserve curated fields such as Asset Group, Attribute, Description, and Main Asset where applicable.
Manual Entry is an SDI exclusion state as well as a review tab. QR_code_assets.Col_process = 2, JSON ExcludeSDI, and QR_codes.sdi = 1 should remain aligned.

App	Script	Port	Image Sequences
ME	`asset_plate_reviewer.py` (1,274 lines)	5002	0, 1, 2, 3
BF	`asset_plate_reviewer_bf.py` (1,749 lines)	5003	0, 1
EL	`Asset_dashboard_EL.py` (1,795 lines)	5004	0, 1, 2

Dashboard Tabs: New (process=0) → Update (process=1) → Manual Entry (process=2)

Core Flow: JSON + Images → Directory Sync → `QR_codes.db` → Filterable Table → Review Form → Save → Approved=1

3.4. Dashboard (:8002)

Current Analytics Behavior

Operational Cost Analysis now uses a merged Data Quality Comparison chart instead of separate completeness and AI confidence cards.
The chart compares weighted completeness and mean AI confidence by asset type and supports All and Open Process scope directly in the chart header.
Dashboard analytics should recompute discipline-aware completeness from current JSON or curated state rather than rely on stale fixed field counts.

Attribute	Value
Main Script	`Asset_portal_dashboard.py` (2,249 lines)
Pattern	Single Page Application (SPA) – `showView()` DOM swapping
Charts	Matplotlib (`Agg` backend) → ephemeral PNG streams

Views: Main Dashboard, Reviewer Analysis, QR Pending, Operational Cost, FLS Assets, User Activity

3.5. SDI Process

Current SDI Rules

SDI Process packages approved curated rows from sdi_dataset and sdi_dataset_EL, then excludes any asset where QR_codes.sdi = 1.
QR enrichment joins must use normalized string QR keys. Placeholder QR IDs such as None, nan, and blank values must be rejected and must not participate in unpackaged or packaged views.

Attribute	Value
Main Script	`app.py` (1,110 lines)
Validation	`asset_template_validation_ver00.py`
Pattern	Tabbed UI – Unpackaged / Packaged

Core Flow: `QR_codes.db` → Unpackaged Assets → Create Package → Export `.xlsx` → Planon Validation → Archive

3.6. Auth Service

File	Purpose
`auth_controller.py`	Flask-Login initialization + user loader
`auth_model.py`	SQLAlchemy models + bcrypt hashing
`init_db.py` / `reset_password.py` / `fix_user.py`	CLI user management scripts

Integration: `sys.path` import → `init_app(app)` on `db` , `bcrypt` , `login_manager`

3.7. Asset Dictionary

Attribute	Value
Core File	`mechanical_dictionary.py` – `ASSET_DICTIONARY` constant
Web App	`dictionary_app.py` – CRUD via AST-safe parsing
Key Format	Composite: `TAG|TYPE` (e.g., `AHU|ME` , `PNL|EL`)
Security	`ast.literal_eval()` – no `eval()` or `exec()`

4. Coding Standards

4.1. Capture App

4.1.1. Flask Routes

- Every non-auth route must have `@login_required`
- Use `request.form.get('param', '')` for POST, `request.args.get('param', '')` for GET
- Validate QR code format (10-digit numeric or `TEMP-XXXXX`) before DB operations

4.1.2. Database Access

- Use `sqlite3.connect()` with `try/finally` and explicit `conn.close()`
- Use the `_open_db()` helper for `QR_codes.db`
- Never hardcode paths, use `os.getenv("QR_CODES_DB_PATH", default_path)`
- Parameterized queries (`?`) only – never concatenate user input into SQL

4.1.3. File Operations

- Filename pattern: `<QR> <Building> <AssetType> - <Seq>.jpg`
- Always sanitize via `sanitize_component()` – strip non-alphanumeric, truncate
- Use `save_image_file()` for uploads – Pillow optimization with fallback
- When renaming: backup first → rename atomically → rollback on failure

4.1.4. Frontend

- **Vanilla ES6+ only** – no frameworks
- Touch targets: minimum 44px × 44px
- Use `ImageCapture` API for photos; fall back to `<canvas>` capture
- Always stop media tracks when leaving camera: `stream.getTracks().forEach(t => t.stop())`

4.2. Extraction API

4.2.1. Architecture Pattern

- Every script follows `AssetProcessor` OOP: `__init__`, `discover_assets()`, `process_single_asset()`, `run()`
- Configuration centralized in `Config` class – no loose globals
- Pydantic models enforce `extra="forbid"` and `str_strip_whitespace=True`
- Entry point: `if __name__ == "__main__":` with `try/except`

4.2.2. Shared Validators (`validators_shared.py`)

- All field normalization MUST live here – never duplicate
- Import via `try/except ImportError` with local fallbacks
- Return empty string `""` on invalid input – never `None`, never raise
- Max lengths: serial=64, model=80, description=120

4.2.3. OCR & Image Processing

- **Hybrid OCR**: Tesseract provides context text injected into LLM prompt
- OCR mode configurable via `*_OCR_MODE` env var: `off`, `light`, `full`
- **Digital Rotation**: 90° CW/CCW preprocessing for vertical text
- **Hallucination Defense (ME)**: UBC Tag regex guard, serial negative prompts, tag-vs-model guardrail

4.2.4. Concurrency

- Use `ThreadPoolExecutor` with configurable `MAX_WORKERS` (default 1, max 2)
- Use `as_completed()` for result collection

4.2.5. Error Handling

- Retry on `APIConnectionError`, `RateLimitError`, `APIStatusError` with configurable retries/delay
- Catch `BadRequestError` separately – do NOT retry
- Catch `ValidationError` from Pydantic – log and return `None`, don't crash batch

---

4.3. Dashboard

Flask Routes

- Every route under `main_bp` must have `@login_required`
- Return chart images via `Response(data, mimetype='image/png')`

4.3.1. Chart Modules

- Place in `charts/` with exported `render_chart_png()` returning `bytes`
- Wrap import with `try/except` and `*_AVAILABLE` boolean flag
- Use `matplotlib.use("Agg")` — never `plt.show()`
- Always `plt.close(fig)` after buffer save

4.3.2. Frontend

- SPA in `dashboard.html` — use `showView('view-id')` for navigation
- CSS custom properties from `responsive-design.css`
- **Never nest** view `div` s inside each other

4.4. Review Apps

4.4.1. Directory Sync

- `sync_image_directory_to_db` and `sync_json_directory_to_db` run on `before_request` — never remove this hook
- Use processed log files to avoid re-processing on every request
- Use `Lock()` for thread-safe sync in multi-worker deployments

4.4.2. JSON Operations

- Read with `encoding='utf-8'` ; write with `ensure_ascii=False, indent=4`
- Set `content["modified"] = True` on any field change
- Never overwrite with lower-quality data – check completeness first

4.4.3. Filtering Rules

1. **Dual Tag Key Check** — always check both `UBC Asset Tag` and `UBC Tag` :

```
if filter_tag:
    data = [i for i in data
            if filter_tag in (i.get("UBC Asset Tag") or i.get("UBC Tag") or "").upper()]
```

2. **Context-Aware Approved Defaults:**

- Process 0 (New) / Process 1 (Update): default `approved_filter` to `"False"`
- Process 2 (Manual Entry): default to `""` (show All)

3. **Navigation Persistence:** `save_review()` must extract `dashboard_query` from POST form, parse into params, pass as kwargs to `url_for()` .

4. **Archive Filtering:** Hide assets in `sdi_print_out_arch` by default; show only with `?archive=false` .

4.4.4. QR Code Renaming

- Only temporary QRs (`T*`) can be renamed
- New code must NOT start with `T` , must be alphanumeric only
- Check conflicts: existing JSON, `QR_codes` , `QR_code_assets`
- Atomic rename: JSON → processed log → images → all DB tables
- Tables to update: `QR_codes` , `QR_code_assets` , `sdi_dataset` , `sdi_dataset_EL` , `sdi_print_out` , `sdi_print_out_arch` , `process_type` , `json_files`

4.4.5. Consistency Across ME / EL / BF

1. Check if change applies to all three apps
2. Use same function names for equivalent operations
3. Keep dictionary lookup priority identical: exact composite → prefix composite → legacy simple
4. Keep image sequence tags consistent: ME `-0,-1,-2,-3` , EL `-0,-1,-2` , BF `-0,-1`

4.5. SDI Process

4.5.1. Database Connections

- Use `with sqlite3.connect(DB_PATH, timeout=10) as conn:`
- Use `table_exists()` checks – archive tables might not exist on fresh DB
- Parameterized queries with `?` placeholders

4.5.2. Multi-Table Exclusions

An asset is "Packaged" if in **either** `sdi_print_out` OR `sdi_print_out_arch` :

```
excluded = get_codes_in_print_out_table().union(get_codes_in_archive_table())
```

4.5.2. Excel Template

- Use Pandas for column renaming via `COLUMN_RENAME_MAP`
- "Panels" → hardcode `EL.21.306.4067` ; otherwise join against `Asset_Group` table

4.5.3. Validation Script (`asset_template_validation_ver00.py`)

- Load with `pd.read_excel(file_path, header=None)` (no type inference)
- Errors → `errors_{base_name}.json` ; Clean → `{base_name}.json`
- Cross-field rules go in `#  - 3. Cross-Field Validation  -`

---

5. Workflows

5.1. Capture → Extraction

1. Navigate to `:5001` → obtain QR (label or TEMP generator)
2. Select Building → Select Asset Class
3. Capture photos (0-Nameplate, 1-Context)
4. `POST /submit` → transactional insert to `QR_code_assets`
5. Trigger extraction: Dashboard Task Manager or `auto_process_assets.sh`
6. `AssetProcessor` consumes images → Hybrid OCR+LLM → JSON written to `Output_jason_api/`

Artifacts: `<QR> <Bldg> <TYP> - 0.jpg, <QR>_<TYP>_<Bldg>.json`

5.2. Batch Extraction

1. Script boots → `Config` class → resolves Windows/Linux paths
2. Spawns `ThreadPoolExecutor` (configurable workers)
3. Files pass through Tesseract → context injected into LLM prompt
4. Computes `completeness_score()` → if acceptable, JSON overwrite
5. Updates `ai_status=1` in database

5.3. Reviewing Extracted Data

1. Login to Review App (e.g., `:5002`)
2. `before_request` hook syncs images/JSONs to `sdi_dataset`
3. Dashboard shows "New" datasets (process=0)
4. Engineer reviews form, edits fields against dictionary
5. Engineer validates against image viewer
6. Clicks "Approve" → `Approved=1` in DB

5.4. SDI Packaging & Compliance

1. Dispatch `/export` → group QR codes into package (e.g., `SDI-00101`)
2. DataFrame logic maps dictionary values to Planon headers
3. `asset_template_validation_ver00.py` verifies cross-field dependencies
4. Delivers finalized `.xlsx` + confirmation summary
5. Archived rows move to `sdi_print_out_arch`

5.5. Atomic Rename Operations

1. `POST /api/update-parameters` → detect property diffs
2. Snapshot backups for `Capture_photos_upload/` files
3. Execute `os.rename()` for all associated files
4. Cascade string updates across all DB tables
5. Update JSON interior (`qr_code` field, `modified=True`)
6. On failure: refuse commit → reverse rename from snapshots

---

6. Special Processes

6.1. The Completeness Guard

Current Guard Rules

The guard is discipline-aware. Do not use one fixed field count across ME, BF, and EL. Mechanical is conditional because Technical Safety BC only counts when sequence -3 exists.
This guard protects extraction overwrites only. Human review saves may bypass the guard and remain the curated source of truth.

Problem: AI vision models are non-deterministic, where reruns may produce worse results.

Solution: Mathematical heuristic preventing overwrites of good data:

1. **New extraction complete** → calculate `Score A`
2. **Check existing JSON** in `Output_jason_api/`
3. **Compare:**
   - No file exists → **Write & Save**
   - File exists → calculate `Score B` from existing file
4. **Gate:**
   - `Score A ≥ Score B` → **Overwrite** (optimistic upgrade)
   - `Score A < Score B` → **SKIP** (log warning)
5. **CRITICAL:** Set `ai_status=1` regardless, it prevents infinite reprocessing loops

Review App Override: Human "Save" sets `modified=True`, bypassing the guard.

6.2. Atomic Rename & Parameter Update

Problem: File names are coupled to parameters (`<QR> <Building> <TYPE>`). Partial updates corrupt state.

Solution: `parameter_update_service.py` implements a 5-step atomic engine:

Step	Action	Rollback
1	**Snapshot** – copy target files to `_temp_rollback` memory	—
2	**OS Rename** – `os.rename()` all associated files	Reverse rename from snapshots
3	**DB Update** – cascade across `QR_codes` , `QR_code_assets` , `sdi_dataset` , `json_files`	Refuse `commit()`
4	**JSON Interior** – update `qr_code` string, set `modified=True`	Revert file content
5	**Commit** – `.commit()` DB + purge rollback list	Full rollback on any exception

6.3. AST Dictionary Parsing

Problem: Dictionary stored as Python source (`.py`) — web editing introduces RCE risk.

Solution: `ast` module for safe read/write:

Operation	Method	Security
Read	`ast.parse()` → find `ASSET_DICTIONARY` assignment → `ast.literal_eval()`	Sandboxed – no code execution
Write	`json.dumps()` + prepend `ASSET_DICTIONARY =`	Always syntactically valid
Key Migration	Legacy `AHU` → composite `AHU|ME` on every save	Prevents cross-type collisions

	Project's approach ☑
Method	ast.literal_eval()
RCE possible?	**No**
Status	**Already secure**

7. Database Schema

7.1. Databases

Database	Owner	Purpose
`User_control.db`	Auth Service	User accounts, bcrypt passwords, roles
`QR_codes.db`	Shared	Asset tracking, AI status, reviews, SDI packaging

`User_control.db`

Table	Key Columns
User	`id` (PK), `username` (unique), `password` (bcrypt), `role`

No foreign keys to `QR_codes.db` — user activities logged by raw username strings to prevent cross-database locks.

7.2. QR_codes.db — Core Tables

A. Asset Registry

Table	Purpose	Key Columns
QR_codes	Master tag list	`QR_code_ID` (PK), `Building_code`, `Location`, `Asset_Type`, `ai_status` (0=Pending, 1=Extracted), `sdi`
QR_code_assets	Human interactions	`id` (PK), `code_assets` (FK→QR_codes), `Col_process`, `user`, `date_hour`

B. SDI Pipeline

Table	Purpose	Key Columns
sdi_dataset	Approved ME/BF assets	`doc_id` (PK), `qr_code` (FK→QR_codes)
sdi_dataset_EL	Approved EL assets	`doc_id` (PK), `qr_code` (FK→QR_codes)
sdi_print_out	Staging packages	`id_print_out` (package ID), `qr_code` (FK→QR_codes)
sdi_print_out_arch	Archived packages	Same as `sdi_print_out`
sdi_sequence	Atomic counter for package names (e.g., `SDI-00101`)	

C. Support Tables

Table	Purpose
json_files	Tracks filenames for atomic renames
process_type	Enum mapping (1=Mechanical, 2=Electrical)

7.3. Entity-Relationship Diagram

```
erDiagram

    User_control_db ||--o{ QR_code_assets : "Logs Username"

    QR_codes ||--o{ QR_code_assets : "Tracks human events"
    QR_codes ||--o{ sdi_dataset : "ME/BF Approvals"
    QR_codes ||--o{ sdi_dataset_EL : "EL Approvals"
    QR_codes ||--o{ json_files : "Tracks physical files"

    sdi_dataset ||--o{ sdi_print_out : "Packaged into"
    sdi_dataset_EL ||--o{ sdi_print_out : "Packaged into"

    sdi_print_out ||--o{ sdi_print_out_arch : "Archived to"

    QR_codes {
        string QR_code_ID PK
        string Building_code
        string Location
        string Asset_Type
        int ai_status
    }

    QR_code_assets {
        int id PK
        string code_assets FK
        string user
        datetime date_hour
    }

    sdi_dataset {
        string doc_id PK
        string qr_code FK
    }

    sdi_dataset_EL {
        string doc_id PK
        string qr_code FK
    }

    sdi_print_out {
        string id_print_out PK
        string qr_code FK
    }

    sdi_print_out_arch {
        string id_print_out PK
        string qr_code FK
    }
```

8. Output Schemas

8.1. JSON Output Envelope

Current Envelope Notes

Current outputs should include the top-level keys qr_code, building_number, asset_type, structured_data, completeness_score, confidence_scores, Avg_ai_conf, and modified where applicable.
Review may add curated fields such as Asset Group, Attribute, Description, Main Asset, ExcludeSDI, Approved, and Flagged after extraction.

All extraction outputs follow this structure:

```
{
"qr_code": "0000184533",
"building_number": "217",
"asset_type": "- ME",
"structured_data": {    . },
"completeness_score": 80.0
}
```

8.2. Structured Data by Asset Type

8.2.1. Mechanical (ME) — `MEStructuredExtraction`

Field	Type	Normalization
Manufacturer	string	Regex brand matching, title-case
Model	string	FCU correction, cap 80 chars
Serial Number	string	Alphanumeric cleanup, cap 64 chars
Year	string	4-digit in 1950-2026, handles MM/YY
UBC Tag	string	FC-prefix grammar, cap 80 chars
Technical Safety BC	string	—

8.2.2. Electrical (EL) — `ELStructuredExtraction`

Field	Type	Normalization
UBC Asset Tag	string	PNL- prefix, abbreviation expansion
Branch Panel	string	—
Ampere	string	Digits + `A` suffix
Volts	string	Compound voltage parsing (e.g., `120/208V`)
Fed / Fed From	string	Whitespace normalization
Location	string	—

8.2.2. Backflow (BF) — `BFStructuredExtraction`

Field	Type	Normalization
Manufacturer	string	Known brand matching (Watts, Wilkins, etc.)
Model	string	Strip special chars, cap 80 chars
Serial Number	string	Alphanumeric cleanup, cap 64 chars
Diameter	string	Fractional inches, ensure `"` suffix

8.3. File Naming Conventions

Type	Pattern	Example
Input images	`<QR> <Building> <Type>-<Seq>.jpg`	`0000183710 100 EL-1.jpg`
Output JSON	`<QR>_<Type>_<Building>.json`	`0000183710_EL_100.json`
Env files	`OpenAI_key_<user>.env`	`OpenAI_key.env`
Shell scripts	`snake_case.sh`	`auto_process_assets.sh`
Python scripts	`API_interface_<TYPE>_ver00.py`	`API_interface_ME_ver00.py`

Image Filename Regex

```
^([T]?\d+)\s+(\d+(?:-\d+)?)\s+([A-Z]{2})\s*-\s*([0-3])$
```

Groups: `QR_code` , `Building_code` , `Asset_type` , `Sequence_number`

8.4. Security Checklist

- ☐ All main routes have `@login_required`
- ☐ No raw SQL concatenation – parameterized queries only
- ☐ No sensitive data in source code — use "`.env`"
- ☐ QR input validated (10-digit numeric or `TEMP-XXXXX`)
- ☐ Uploaded filenames are server-generated
- ☐ File paths validated – no path traversal
- ☐ Camera access only over HTTPS/localhost
- ☐ `ast.literal_eval()` for dictionary parsing – never `eval()`